Shell Script Assignment

Q1. Write a shell script to print arguments passed to the script in reverse order.

Algorithm / Steps:

1. Start the script.
2. Accept arguments from the command line.
3. Loop through arguments in reverse order.
4. Print each argument.
5. Stop the script.

Program:

```
#!/bin/bash
for (( i=$#; i>0; i-- ))
do
    echo "${!i}"
done
```

Output:

```
[ec2-user@ip-172-31-23-167 ~]$ bash Assignment/AQ1.sh 1 2 3 4 5
5
4
3
2
1
[ec2-user@ip-172-31-23-167 ~]$
```

Q2. Write a script that monitors system performance and sends mail alert.

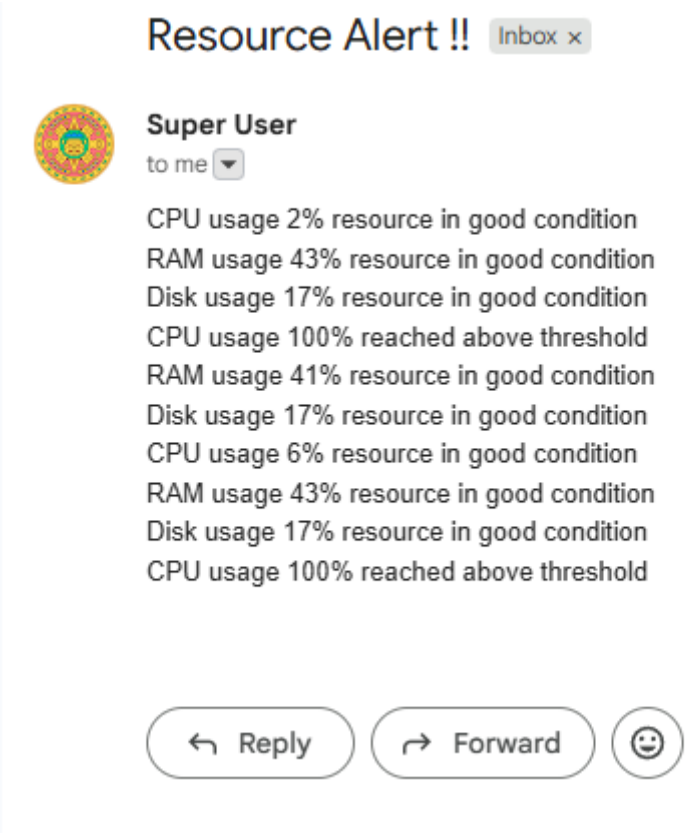
Program:

```
#!/bin/bash

cpu_ug=$(mpstat 1 1 | awk '/Average:/ {printf("%.0f", 100 - $NF)}')
mem_ug=$(free | awk '/Mem/ {printf("%.0f", $3/$2 * 100)}')
dsk_ug=$(df / | awk 'NR==2 {print $5}' | sed 's/%//g')
logfile="/var/log/resources_alert.log"
mailid="narendrageddam95@gmail.com"

for res_typ in CPU RAM Disk
do
        case $res_typ in
                CPU) res=$cpu_ug;;
                RAM) res=$mem_ug;;
                Disk) res=$dsk_ug;;
        esac

        if (($res >= 80))
        then
                echo "$res_typ usage $res% reached above threshold" >>
$logfile
                echo "$res_typ usage $res% reached above threshold"
                tail -n 10 $logfile | mail -s "Resource Alert !!"
"$mailid"
        else
                echo "$res_typ usage $res% resource in good condition"
>> $logfile
                echo "$res_typ usage $res% resource in good condition"
        fi
done
```

Output:

```
[ec2-user@ip-172-31-23-167 ~]$ sudo bash Assignment/AQ2.sh
CPU usage 6% resource in good condition
RAM usage 43% resource in good condition
Disk usage 17% resource in good condition
[ec2-user@ip-172-31-23-167 ~]$ sudo bash Assignment/AQ2.sh
CPU usage 100% reached above threshold
RAM usage 43% resource in good condition
Disk usage 17% resource in good condition
[ec2-user@ip-172-31-23-167 ~]$
```

Mail received on reached above threshold:



Q3. Difference between ./script and nohup ./script &

Explanation:

- `./script` → runs in foreground, stops when terminal closes.
- `nohup ./script &` → runs in background, ignores hangup, continues after logout.

Result:
We understood the difference between foreground and nohup background execution.

Q4. Read an integer 'n' and generate the following pattern:

1

1 2

1 2 3

1 2 3 4

up to 'n' rows.

Program:

```
#!/bin/bash
read -p "Enter n: " n
for ((i=1; i<=n; i++))
do
    seq -s " " 1 $i
done
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ3.0.sh
Enter a number: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
[ec2-user@ip-172-31-23-167 Assignment]$
```

Q5. Write a script to display the contents of the file in reverse order without using tac command.

Program:

```
#i!/bin/bash

read -p "Enter file name: " fname

i=0
while read  line
do
    arr[i]="$line"
    ((i++))
done < "$fname"

for ((j=i-1; j>=0; j--))
do
    echo "${arr[j]}"
done
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ cat data.txt
Apple
Banana
Cherry
Dates
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ4.sh
Enter file name: data.txt
Dates
Cherry
Banana
Apple
[ec2-user@ip-172-31-23-167 Assignment]$ cat AQ4.sh
```

Q 6. Write a script to reverse a string using while loop

Input: hello

Output: olleh

Program:

```
#!/bin/bash

read -p "Enter the string : " str
len=$(echo $str | wc -c)
n=$(($len-1))
rev=""

while [ $n -ge 0 ]
do
        rev="$rev${str:$n:1}"
        ((n--))
done
echo "$rev"
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ5.sh
Enter the string : RedHat
taHdeR
[ec2-user@ip-172-31-23-167 Assignment]$
```

Q7. Read an integer 'n' and generate the following pattern:

1

2 3

4 5 6

7 8 9 10

up to 'n' rows."

Program:

```
#!/bin/bash
read -p "Enter n: " n
num=1
for ((i=1; i<=n; i++))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "$num "
        ((num++))
    done
    echo
done
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ6.sh
Enter n number: 6
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
[ec2-user@ip-172-31-23-167 Assignment]$
```

Q8. Write a script that renames a file or directory by replacing its letters with lowercase from uppercase letters.

Example: TEst.txt → test.txt

Program:

```
#!/bin/bash
read -p "Enter file or directory name: " name
lower_name="${name,,}"
mv "$name" "$lower_name"
echo "Renamed: $name → $lower_name"
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ ls HELlo.txt
HELlo.txt
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ7.sh
Enter file or directory name: HELlo.txt
Renamed: HELlo.txt → hello.txt
[ec2-user@ip-172-31-23-167 Assignment]$ ls hello.txt
hello.txt
[ec2-user@ip-172-31-23-167 Assignment]$ ls HELlo.txt
ls: cannot access 'HELlo.txt': No such file or directory
[ec2-user@ip-172-31-23-167 Assignment]$
```

Q12. Write a script that takes multiple inputs from the user as arguments and only prints the arguments if they are integers.

Input: sh script.sh abc 2 t5 4 efg ghi 34 67 1 2

Output The integer arguments are:

2

4

34

67

1

Program:

```
#!/bin/bash
echo "The integer arguments are:"
for arg in "$@"
do
    if [[ "$arg" =~ ^[0-9]+$ ]]
    then
        echo "$arg"
    fi
done
```

Output:

```
[ec2-user@ip-172-31-23-167 Assignment]$ bash AQ8.sh \
> 234 567 890 qwe rty ui 34 rft
234
567
890
34
[ec2-user@ip-172-31-23-167 Assignment]$
```